



UNIVERSIDAD DE GRANADA

ETSIIT

Escuela Técnica Superior
de Ingenierías Informática
y de Telecomunicación



INGENIERÍA DE COMPUTADORES, AUTOMÁTICA Y ROBÓTICA

SERVIDORES WEB DE ALTAS PRESTACIONES

Tema: TRABAJO FINAL: GRANJA DE MODELOS IA

Autor: FRANCISCO JAVIER PÉREZ GARCÍA, INGMATFCO@CORREO.UGR.ES, A2

Autor: MANUEL RUIZ AGUILAR, MANUEL100@CORREO.UGR.ES, A3

Autor: GABRIEL FRANCISCO SÁNCHEZ MUÑOZ, GABRIELFSM@CORREO.UGR.ES, A3

Profesor: JOSÉ MANUEL SOTO HIDALGO, JMSOTO@UGR.ES

Profesora: MIRIAM MENGÍBAR RODRÍGUEZ, MIRISMR@UGR.ES

Índice

Listado de imágenes	0
1. Introducción	1
2. Fundamentos teóricos	1
2.1. Docker y Docker Compose	1
2.2. Ollama	2
2.3. LiteLLM	2
2.4. Redis Stack y caché semántico	2
2.5. Nginx como proxy inverso	3
2.6. Locust y Python	3
3. Objetivos	3
4. Arquitectura del sistema	4
4.1. Diagrama de arquitectura	4
4.2. Descripción de componentes	4
5. Implementación	6
5.1. Orquestación con Docker Compose	6
5.2. Configuración del gateway LiteLLM	7
5.3. Configuración del proxy Nginx	8
6. Análisis de rendimiento	9
6.1. Metodología de evaluación	9
6.2. Grupos experimentales	9
6.3. Resultados	10
6.3.1. Test LiteLLM	10
6.3.2. Locust	11
6.4. Análisis de los resultados	11
7. Conclusiones	13
7.1. Cumplimiento de objetivos	13
7.2. Limitaciones	13
7.3. Trabajo futuro	13
Bibliografía	15

Listado de imágenes

1. Arquitectura del sistema: flujo de peticiones entre componentes.	4
2. Gráficos, primera prueba.	10
3. Tiempos según percentiles de grupos experimentales en una gráfica.	11

Trabajo final: Granja de Modelos IA

1. Introducción

Los grandes modelos de lenguaje (LLM, por sus siglas en inglés) han transformado la manera en que interactuamos con la información. Sin embargo, servirlos de forma eficiente presenta desafíos significativos: la latencia de inferencia es alta (en CPU puede superar los 6 segundos por petición), una única instancia es un punto único de fallo, y sin mecanismos de memorización se repite trabajo ya realizado ante preguntas idénticas o similares.

En este trabajo abordamos estos problemas construyendo una granja de modelos de IA que combina tres pilares fundamentales: *replicación de instancias de inferencia*, *balanceo de carga* y *caché semántico*. El sistema se despliega mediante Docker Compose y está compuesto por seis servicios: dos instancias de Ollama para inferencia con el modelo `qwen3.5:0.8b`, una instancia de Ollama dedicada a embeddings con `nomic-embed-text`, un gateway LiteLLM que actúa como punto de enrutamiento y gestor de caché, Redis Stack como almacén del caché semántico vectorial, y Nginx como proxy inverso de entrada.

La novedad principal del trabajo reside en el uso de caché semántico: en lugar de limitarnos a servir respuestas idénticas desde caché (caché exacto), empleamos Redis con búsqueda por similitud coseno sobre embeddings para detectar preguntas parafraseadas y devolver la respuesta almacenada. Los resultados muestran una aceleración de $191.5\times$ en latencia para aciertos de caché (31 ms frente a 5929 ms) y una distribución de carga equilibrada entre las instancias de inferencia.

El resto del documento se organiza como sigue: la Sección 2 presenta los fundamentos teóricos de cada tecnología empleada; la Sección 3 enumera los objetivos del proyecto; la Sección 4 describe la arquitectura del sistema; la Sección 5 detalla la implementación; la Sección 6 analiza el rendimiento mediante experimentos; y la Sección 7 recoge las conclusiones y líneas de trabajo futuro.

2. Fundamentos teóricos

2.1. Docker y Docker Compose

Docker [1] es una plataforma de contenerización que permite empaquetar aplicaciones y sus dependencias en unidades ligeras y portables llamadas contenedores. A diferencia de las máquinas virtuales, los contenedores comparten el núcleo del sistema anfitrión, lo que reduce la sobrecarga y acelera el arranque.

Docker Compose [2] extiende Docker para orquestar múltiples contenedores como un servicio unificado. Mediante un archivo `docker-compose.yml` se declaran los servicios, las redes que los comunican y los volúmenes que persisten sus datos. En nuestro proyecto, Docker Compose define una red `bridge` que aísla el tráfico interno y permite la resolución de nombres por nombre de servicio, facilitando que LiteLLM se comunique con los contenedores de Ollama simplemente usando `http://ollama-1:11434`.

2.2. Ollama

Ollama [3] es un *runtime* local para ejecutar modelos de lenguaje de código abierto. Ofrece una API REST compatible con OpenAI que permite descargar, servir e inferir modelos sin necesidad de configuraciones complejas.

Para inferencia utilizamos Qwen3.5 0.8b [4], un modelo de 800 millones de parámetros entrenado por Alibaba. Es ligero (aproximadamente 0.5 GB), compatible con CPU, y ofrece respuestas razonables para tareas de conocimiento general. Para generación de embeddings empleamos `nomic-embed-text` [5], un modelo especializado de 137 millones de parámetros que produce vectores de 768 dimensiones, adecuados para búsqueda por similitud semántica.

Cada instancia de Ollama se ejecuta en su propio contenedor con un volumen Docker dedicado que persiste los modelos descargados, evitando la descarga repetida en cada reinicio. Un script de inicialización (*entrypoint*) se encarga de descargar el modelo correspondiente al arrancar el contenedor.

2.3. LiteLLM

LiteLLM [6] es un *proxy* de inferencia de código abierto que ofrece una API unificada compatible con el formato de OpenAI. Actúa como capa de abstracción sobre múltiples *backends* de modelos (Ollama, OpenAI, Anthropic, etc.) y proporciona funcionalidades clave para entornos de producción:

- **Enrutamiento:** distribuye peticiones entre varios *backends* del mismo modelo lógico según la estrategia configurada. En nuestro caso usamos `simple-shuffle`, que rota las peticiones de forma aleatoria entre las instancias disponibles, logrando un reparto aproximadamente equitativo.
- **Caché semántico:** LiteLLM soporta varios *backends* de caché [7]. El modo `redis-semantic` utiliza Redis Stack para almacenar pares (embedding de la pregunta, respuesta) y, ante una nueva pregunta, compara su embedding con los almacenados usando similitud coseno. Si la similitud supera un umbral configurable (en nuestro caso, 0.85), devuelve la respuesta cacheada sin invocar al modelo.
- **Gestión de errores:** reintentos automáticos ante fallos transitorios del *backend*, con reintentos configurables.

2.4. Redis Stack y caché semántico

Redis Stack [8] extiende Redis con capacidades de búsqueda vectorial a través del módulo Redisearch. Permite crear índices sobre campos vectoriales y ejecutar consultas de los k vecinos más cercanos (KNN) usando métricas como la similitud coseno.

El flujo del caché semántico en nuestro sistema es el siguiente:

1. Ante una pregunta del usuario, LiteLLM solicita al modelo de embeddings (`nomic-embed-text`) el vector correspondiente a la pregunta.
2. Consulta Redis Stack por los k vectores más cercanos en el índice, con un umbral de similitud coseno ≥ 0.85 .
3. Si existe una coincidencia, devuelve la respuesta almacenada en Redis (caché HIT) en aproximadamente 31 ms.
4. Si no hay coincidencia (caché MISS), envía la pregunta al modelo de inferencia, almacena el par (embedding, respuesta) en Redis y devuelve la respuesta generada en aproximadamente 5929 ms.

La similitud coseno [9] entre dos vectores $\vec{a}$ y $\vec{b}$ se define como:

$$\text{sim}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|} \in [-1, 1] \quad (2.1)$$

Dos vectores idénticos tienen similitud 1, vectores ortogonales 0, y vectores opuestos -1. En la práctica, con embeddings normalizados, el umbral 0.85 captura paráfrasis cercanas.

2.5. Nginx como proxy inverso

Nginx [10] es un servidor web y proxy inverso de alto rendimiento. En nuestro sistema actúa como punto de entrada único, exponiendo el puerto 80 al exterior y redirigiendo el tráfico al *upstream* de LiteLLM en el puerto 4000. Se configura con *timeouts* largos (120 s) para tolerar la latencia de inferencia, y con el *buffering* de proxy desactivado para permitir respuestas en *streaming* (SSE, Server-Sent Events) desde el modelo.

2.6. Locust y Python

Locust [11] se trata de una herramienta que permite realizar pruebas de carga realistas para contextos con usuarios, esto con la función de encontrar cuellos de botellas, validar la capacidad del sistema, ... Pero en nuestro contexto lo empleamos para realizar las pruebas de funcionamiento. Es de código abierto y está basado en el lenguaje Python, lo que permite aumentar la complejidad y la versatilidad de las pruebas definiendo *tasks* en un archivo *locustfile.py*. Puede combinarse con Docker en lugar de emplear la librería *subprocess*.

También puede emplearse y empleamos un código alternativo con funcionalidades muy útiles en Python con el que ejecutamos otra prueba, pero por supuesto, el código es más difícil de comprender y modificar, y este no podría extenderse tan fácilmente a realizar pruebas de cargas.

3. Objetivos

Los objetivos del proyecto son los siguientes:

1. Desplegar una granja de modelos de IA funcional y reproducible utilizando Docker Compose, con separación de responsabilidades entre proxy, *gateway*, inferencia y almacenamiento de estado.
2. Implementar balanceo de carga entre múltiples instancias de inferencia Ollama para mejorar la disponibilidad y distribuir la carga de trabajo.
3. Incorporar un sistema de caché semántico basado en Redis Stack que detecte preguntas parafraseadas — no solo duplicados exactos — y devuelva respuestas almacenadas sin necesidad de ejecutar inferencia.
4. Exponer el sistema completo tras un proxy Nginx unificado, accesible a través de un único puerto, con soporte para *streaming* de respuestas.
5. Medir y analizar cuantitativamente el rendimiento del sistema: tasa de acierto de caché, latencia con y sin caché, distribución de carga entre instancias, y casos límite del caché semántico (falsos positivos y falsos negativos).

4. Arquitectura del sistema

4.1. Diagrama de arquitectura

La Figura 1 muestra la arquitectura global del sistema. Las peticiones de los clientes entran a través de Nginx (puerto 80), que las redirige al *gateway* LiteLLM (puerto 4000). LiteLLM consulta Redis para determinar si la pregunta está cacheada y, en caso de no estarlo, enruta la petición a una de las dos instancias de inferencia mediante la estrategia **simple-shuffle**. La instancia de embeddings es consultada tanto por LiteLLM (para generar el vector de la pregunta entrante) como por Redis (para comparar con los vectores almacenados en el índice).

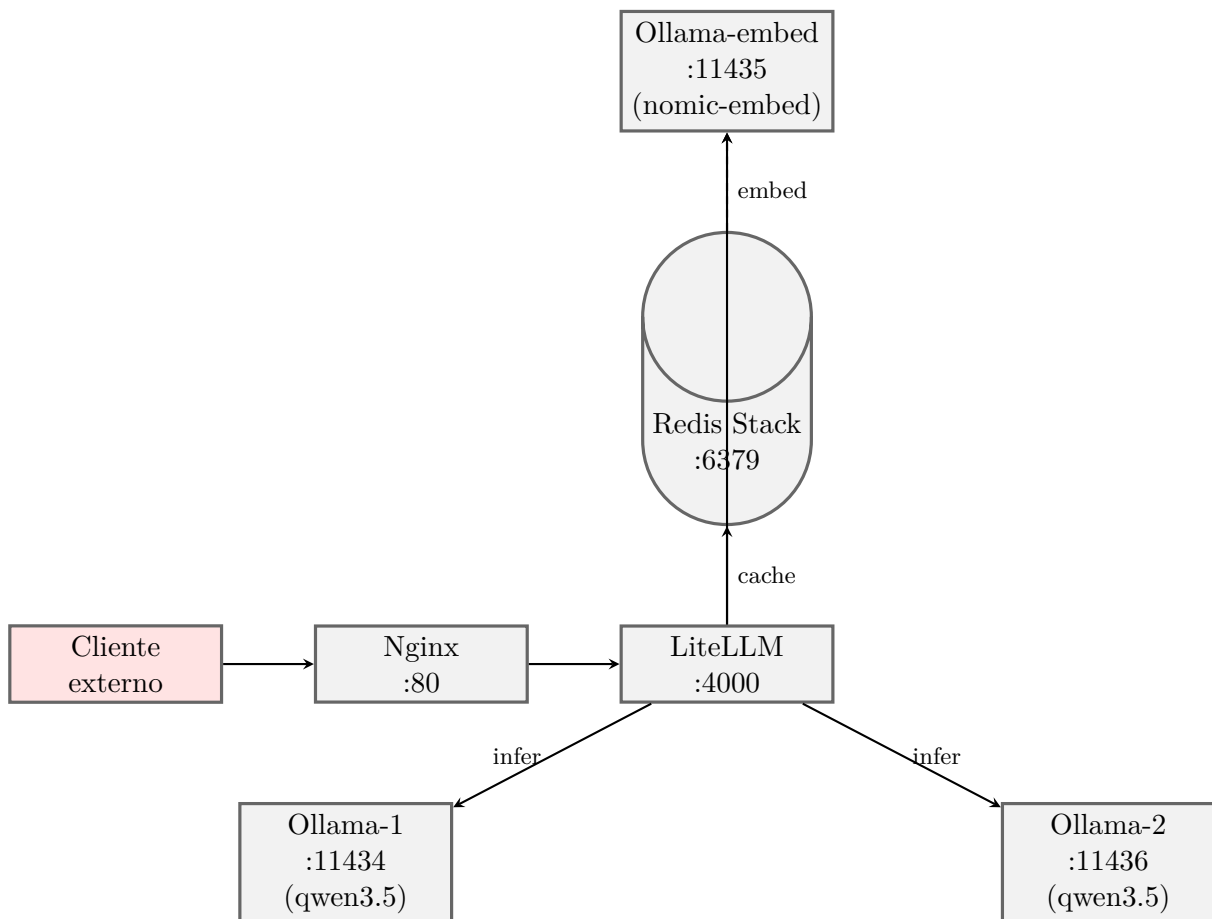


Figura 1: Arquitectura del sistema: flujo de peticiones entre componentes.

4.2. Descripción de componentes

Nginx actúa como *reverse proxy* de entrada. Recibe peticiones HTTP en el puerto 80 y las reenvía al *upstream* `litellm_backend`, definido como el servicio LiteLLM en el puerto 4000. El *buffering* de proxy está desactivado para permitir que las respuestas en *streaming* (SSE) lleguen al cliente sin retraso. Los *timeouts* de conexión, envío y lectura se extienden a 120 segundos para acomodar la latencia de inferencia.

LiteLLM es el núcleo del sistema. Expone una API compatible con OpenAI en el puerto 4000. Su configuración declara dos *backends* lógicos con el mismo nombre de modelo (**qwen3**), cada uno apuntando a una instancia de Ollama distinta. La estrategia de enrutamiento **simple-shuffle** distribuye las peticiones aleatoriamente. Además, LiteLLM gestiona el caché semántico: ante cada petición, genera el embedding de la pregunta, consulta Redis por el vector más cercano y decide si responde desde caché o delega en el modelo.

Ollama-1 y Ollama-2 son instancias idénticas de inferencia que ejecutan el modelo **qwen3.5:0.8b**. Cada una expone su API en un puerto distinto del *host* (11434 y 11436 respectivamente), aunque internamente ambas escuchan en el 11434. Utilizan volúmenes Docker independientes para la persistencia del modelo.

Ollama-embed es una instancia de Ollama dedicada exclusivamente a generar embeddings con el modelo **nomic-embed-text**. Es consultada por LiteLLM cada vez que se necesita vectorizar una pregunta entrante para la búsqueda en caché. Su separación en un contenedor independiente evita interferencias con la carga de inferencia y permite escalar los servicios de forma independiente.

Redis Stack actúa como almacén del caché semántico, puesto que es una extensión de la base de datos Redis que facilita tratar datos complejos como estos. LiteLLM, a través de su módulo **RedisSemanticCache**, crea automáticamente un índice vectorial en Redis y almacena las respuestas generadas. Ante búsquedas de similitud, Redis devuelve la respuesta más cercana si supera el umbral anteriormente mencionado.

Red semantic-cache-net es una red *bridge* de Docker que conecta todos los servicios. Permite la resolución DNS interna por nombre de contenedor (ej. **redis**, **ollama-1**), lo que simplifica la configuración y aísla el tráfico del sistema del resto del *host*.

5. Implementación

5.1. Orquestación con Docker Compose

El archivo `docker-compose.yml` (Listado 1) define los cinco servicios, cuatro volúmenes con nombre y la red `semantic-cache-net`. Las dependencias entre servicios se modelan con `depends_on`, garantizando que Redis y las instancias de Ollama estén operativas antes de que LiteLLM se inicie.

Listing 1: Definición de servicios en `docker-compose.yml`

```
services:
  nginx:
    image: nginx:alpine
    container_name: semantic-cache-nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - litellm
    networks:
      - semantic-cache-net
    restart: unless-stopped

  litellm:
    image: ghcr.io/berriai/litellm:main-latest
    container_name: semantic-cache-litellm
    ports:
      - "4000:4000"
    volumes:
      - ./litellm/configv2.yaml:/app/config.yaml:ro
    environment:
      LITELLM_CONFIG_PATH: /app/config.yaml
      LITELLM_MODE: "production_no_db"
      OLLAMA_API_BASE: "http://ollama-embed:11434"
    command: ["--config", "/app/config.yaml", "--port", "4000"]
    depends_on:
      redis:
        condition: service_healthy
      ollama-embed:
        condition: service_healthy
      ollama-1:
        condition: service_started
      ollama-2:
        condition: service_started
```

Cada instancia de Ollama define un `entrypoint` personalizado (Listado 2) que ejecuta el servidor Ollama en segundo plano y, tras una breve pausa, descarga el modelo requerido mediante `ollama pull`. El modelo queda persistido en un volumen con nombre, por lo que reinicios posteriores del contenedor no necesitan volver a descargarlo.

Listing 2: Script de inicialización de Ollama (inferencia)

```

1 #!/bin/sh
2 set -e
3 echo "[ollama-init] Pulling qwen3.5:0.8b..."
4 ollama pull qwen3.5:0.8b
5 echo "[ollama-init] Done."

```

La instancia de embeddings usa un script análogo que descarga `nomic-embed-text` en lugar del modelo de inferencia.

Redis Stack se configura con un *healthcheck* que verifica la conectividad mediante `redis-cli ping`. Solo cuando Redis y Ollama-embed están sanos, LiteLLM completa su inicialización, ya que depende de ambos para el funcionamiento del caché semántico.

5.2. Configuración del gateway LiteLLM

El archivo `configv2.yaml` (Listado 3) constituye la configuración central del sistema. Define tres entradas en `model_list`: dos para inferencia (ambas bajo el nombre lógico `qwen3`, apuntando a `ollama-1` y `ollama-2` respectivamente), y una para embeddings (`nomic-embed-text` en `ollama-embed`).

Listing 3: `configv2.yaml` — Configuración de LiteLLM

```

model_list:
  - model_name: qwen3
    litellm_params:
      model: ollama/qwen3.5:0.8b
      api_base: http://ollama-1:11434
      timeout: 60
      stream_timeout: 120
  - model_name: qwen3
    litellm_params:
      model: ollama/qwen3.5:0.8b
      api_base: http://ollama-2:11434
      timeout: 60
      stream_timeout: 120
  - model_name: ollama/nomic-embed-text
    litellm_params:
      model: ollama/nomic-embed-text
      api_base: http://ollama-embed:11434

router_settings:
  routing_strategy: simple-shuffle
  num_retries: 2
  retry_after: 5
  timeout: 30

litellm_settings:
  cache: true
  cache_params:
    type: redis-semantic
    redis_url: redis://redis:6379
    similarity_threshold: 0.85
    redis_semantic_cache_embedding_model: ollama/nomic-embed-text
    vector_size: 768
    ttl: 3600

```

```

drop_params: true
request_timeout: 60
telemetry: false

general_settings:
  disable_spend_logs: true
  disable_master_key_hashing: true

```

Los parámetros clave del caché semántico son:

- **type: redis-semantic:** activa el *backend* de caché semántico basado en Redis Stack.
- **similarity_threshold: 0.85:** umbral de similitud coseno. Una nueva pregunta debe tener una similitud ≥ 0.85 con alguna pregunta almacenada para que se devuelva la respuesta cacheada.
- **vector_size: 768:** dimensionalidad de los embeddings producidos por *nomic-embed-text*.
- **ttl: 3600:** tiempo de vida de las entradas en caché (1 hora), que evita el crecimiento ilimitado de la base de datos.

La estrategia de enrutamiento **simple-shuffle** selecciona aleatoriamente entre *ollama-1* y *ollama-2* para cada petición que no se resuelve desde caché. Con **num_retries: 2**, si la instancia seleccionada falla, se reintenta en la otra.

5.3. Configuración del proxy Nginx

El archivo `nginx.conf` (Listado 4) define el *upstream* hacia LiteLLM, los *timeouts* extendidos para inferencia y la configuración para respuestas en *streaming*.

Listing 4: `nginx.conf` — Configuración del proxy inverso

```

events {
    worker_connections 1024;
}
http {
    upstream litellm_backend {
        server litellm:4000;
        keepalive 32;
    }
    proxy_connect_timeout 10s;
    proxy_send_timeout 120s;
    proxy_read_timeout 120s;
    send_timeout 120s;
    server {
        listen 80;
        server_name _;
        location / {
            proxy_pass http://litellm_backend;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header Connection "";
            proxy_buffering off;
            chunked_transfer_encoding on;
        }
    }
}

```

Las directivas clave son `proxy_buffering off`, necesaria para que las respuestas en *streaming* (SSE) fluyan desde el modelo hasta el cliente sin que Nginx las almacene en búfer, y `proxy_read_timeout 120s`, que evita que Nginx cierre conexiones durante generaciones de texto largas.

6. Análisis de rendimiento

6.1. Metodología de evaluación

Para evaluar el rendimiento del sistema se ha desarrollado un *script* de pruebas en Python (`test_litellm.py`) que ejecuta 89 peticiones organizadas en siete grupos experimentales, cada uno diseñado para verificar un aspecto concreto del comportamiento del caché semántico y del balanceo de carga. De forma análoga, empleamos las peticiones para ejecutar Locust y estudiar los tiempos de respuesta.

La detección de aciertos de caché (HIT) y fallos (MISS) se realiza mediante el análisis de los registros (*logs*) del contenedor de LiteLLM. LiteLLM escribe la cadena `Cache Hit!` en su salida estándar de forma síncrona antes de enviar la respuesta HTTP. El *script* toma una instantánea de los logs antes y después de cada petición; si el contador de ocurrencias de `Cache Hit!` se incrementa, la petición se clasifica como HIT, y como MISS en caso contrario. La instancia de Ollama que atendió cada petición se determina contando las líneas que contienen la URL del *api_base* correspondiente (`http://ollama-1:11434` o `http://ollama-2:11434`). Locust, por otra parte, tras haber configurado en su archivo la fracción correspondiente a cada tarea (*@task*); una por grupo; recoge las métricas correspondientes a cada grupo y podremos consultarlas, ya sea a través de la interfaz o mediante archivos `‘.csv’`.

Antes de cada ejecución del *script* se vacía la base de datos Redis y se reinicia LiteLLM para garantizar que el caché parte de un estado limpio y que los índices vectoriales se recrean correctamente. A continuación se realiza un calentamiento con cuatro preguntas triviales que fuerzan la carga inicial de los modelos en memoria.

6.2. Grupos experimentales

Los siete grupos de experimentos y su propósito son:

1. **Duplicados exactos:** 10 temas distintos $\times$ 2 preguntas idénticas. La segunda repetición siempre debe ser HIT (10/10 esperados). Verifica el funcionamiento básico del caché.
2. **Paráfrasis — Geografía:** 5 temas de geografía $\times$ 3 formulaciones parafraseadas. Las reformulaciones 2 y 3 deberían ser HIT (10/10 esperados). Evalúa la capacidad del caché semántico para detectar sinonimia.
3. **Paráfrasis — Ciencia:** 5 temas de ciencia $\times$ 3 formulaciones. Análogo al anterior pero en un dominio con vocabulario más técnico.
4. **Paráfrasis — Historia y Arte:** 5 temas $\times$ 3 formulaciones. Mismo diseño en otro dominio.
5. **Temas relacionados pero distintos:** 3 dominios $\times$ 3 preguntas diferentes sobre el mismo tema. Ninguna debería producir HIT (0/9). Verifica que el umbral 0.85 no es demasiado permisivo con preguntas distintas.
6. **No relacionados:** 5 preguntas de dominios completamente dispares. Ninguna debería ser HIT (0/5). Control cruzado.
7. **Sonda de balanceo de carga:** 10 preguntas idénticas con caché desactivado (`no-cache: true`). Mide la distribución de peticiones entre ollama-1 y ollama-2.

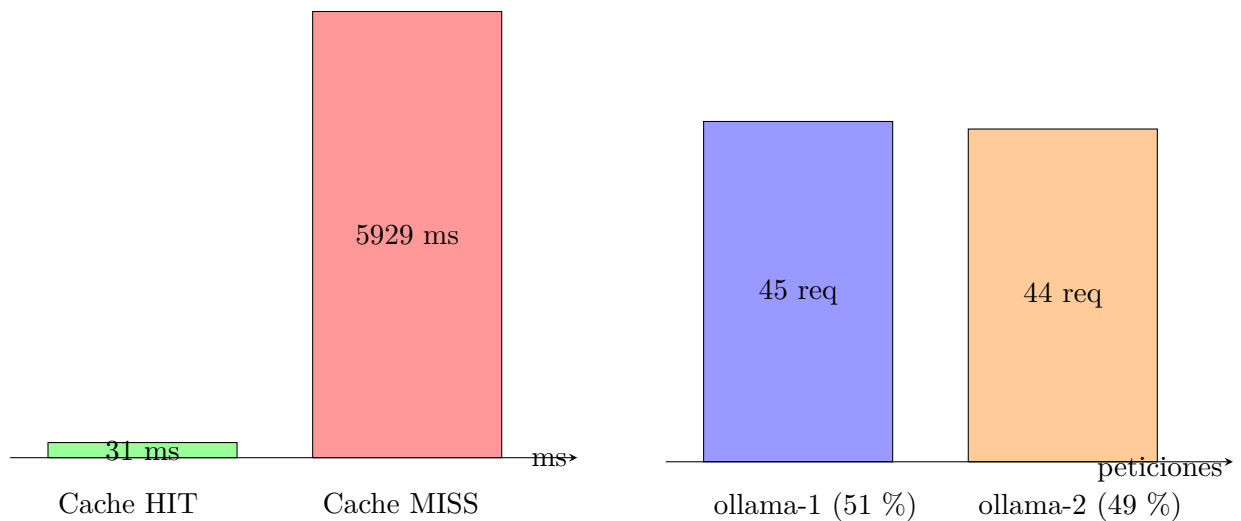
6.3. Resultados

6.3.1. Test LiteLLM

La Tabla 1 resume los resultados agregados por grupo experimental. Las Figuras 2a y 2b visualizan los datos de latencia y distribución de carga respectivamente.

Tabla 1: Resultados agregados por grupo experimental. Se indica el número de HITs esperados y los obtenidos.

Grupo	Peticiones	HITs esperados	HITs reales	Acierto
Duplicados exactos	20	10	10	100 %
Paráfrasis Geografía	15	10	10	100 %
Paráfrasis Ciencia	15	10	4	40 %
Paráfrasis Historia	15	10	7	70 %
Relacionados distintos	9	0	0	0 %
No relacionados	5	0	1	—
Balanceo de carga	10	0	0	—



Factor de aceleración: 191,5×

(a) Latencia media de las peticiones con acierto de caché (31 ms) frente a las que requieren inferencia completa (5929 ms).

(b) Distribución de 89 peticiones entre las dos instancias de inferencia.

Figura 2: Gráficos, primera prueba.

6.3.2. Locust

En la siguiente tabla, vemos el número de peticiones de cada grupo y sus tiempos mínimos, máximos y medios, e incluso el tamaño medio de la respuesta.

Tabla 2: Resultados agregados por grupo experimental en Locust.

Grupo	Peticiones	Mediana RT	Media RT	Mín. RT	Máx. RT	Tamaño medio
Duplicados exactos	129	310	3780	45	52933	308
Balanceo de carga	12	28000	25615	12583	44469	281
Paráfrasis Geografía	24	250	9864	56	52635	304
Paráfrasis Historia	28	880	8357	59	37526	300
Paráfrasis Ciencia	40	250	7619	57	47966	303
Relacionados distintos	23	1400	13803	60	44017	298
No relacionados	15	410	8302	60	59903	305

Donde RT abrevia *Request Time*, los tiempos se miden en milisegundos y el tamaño en bytes. Se han obviado las columnas de fallos, que estaban a cero, y los percentiles los aportamos a modo de gráfica a continuación.

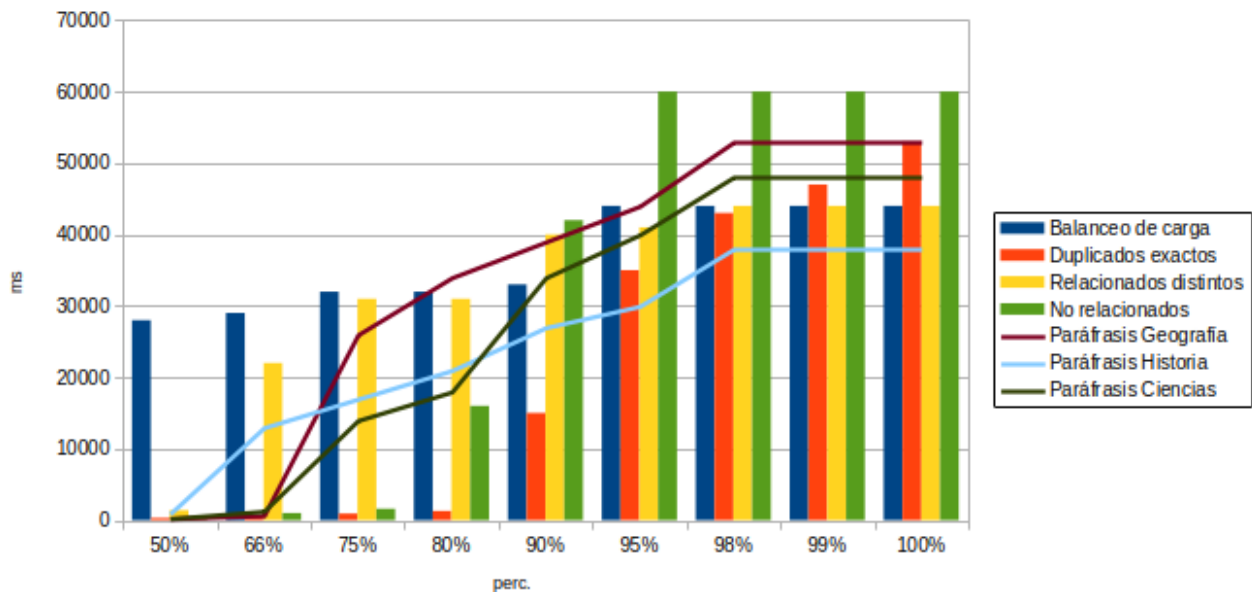


Figura 3: Tiempos según percentiles de grupos experimentales en una gráfica.

6.4. Análisis de los resultados

Caché exacto (100 % de acierto). Las 10 preguntas repetidas de forma idéntica en el grupo de duplicados exactos obtuvieron HIT en su segunda aparición sin excepción. Esto confirma que el flujo básico de almacenamiento y recuperación del caché funciona correctamente. Respecto a sus tiempos, aproximadamente a partir del percentil noventa, donde vemos métricas altas, coincide la proporción de las peticiones por primera vez (10) respecto al total del grupo (129), que están formadas por la mismas diez peticiones repetidas, y sus tiempos de respuesta se reducen con el caché.

Caché semántico sobre paráfrasis. El rendimiento varía según el dominio. En Geografía, las

10 paráfrasis esperadas se detectaron correctamente (100 %). En Historia y Arte, 7 de 10 (70 %); adicionalmente se produjo un falso positivo en la primera pregunta de la Gran Muralla China (*“What is the Great Wall of China?”*), que acertó en caché sin haber sido formulada antes. En Ciencia solo se detectaron 4 de 10 (40 %). Los falsos negativos se concentran en preguntas como las relativas a la fotosíntesis (*“How does photosynthesis work?”* vs. *“Explain the process of photosynthesis”*) o la gravedad (*“What is gravity?”* vs. *“How does gravity work?”*), donde el modelo de embeddings no asigna suficiente similitud coseno a pesar de la equivalencia semántica. Esto sugiere que el umbral 0.85 puede ser demasiado restrictivo para ciertos dominios.

La correlación entre un aumento de las métricas temporales y el número de MISS debería poder apreciarse si todas las peticiones de paráfrasis fueran distintas, pero en este caso (Locust) se repiten y no hay mecanismo para hacer un uso equitativo de estas, y como el número de peticiones realizadas supera el número de peticiones distintas es obvio que se han repetido. Los últimos percentiles (probablemente correspondientes a las primeras veces de cada petición) nos muestran que la dificultad de los temas de algunos grupos de paráfrasis es distinta, ya sea por las preguntas concretas o la configuración del modelo en sí.

Falsos positivos. Se detectó un único falso positivo en el grupo de no relacionados: la pregunta *“What is the boiling point of ethanol?”* obtuvo HIT, probablemente por similitud con la pregunta *“What is the boiling point of water?”* del grupo de duplicados exactos. Este caso ilustra el compromiso inherente al caché semántico: umbrales más bajos mejoran la tasa de acierto en paráfrasis a costa de aumentar los falsos positivos.

Latencia. La diferencia es drástica: las peticiones resueltas desde caché promedian 31 ms, frente a los 5929 ms que requiere la inferencia completa en CPU con el modelo `qwen3.5:0.8b`. El factor de aceleración es de $191.5\times$ (mediana de 31 ms frente a 5925 ms). La latencia de caché está dominada por la generación del embedding de la pregunta entrante y la consulta al índice vectorial de Redis; la latencia de inferencia refleja el coste computacional de generar la respuesta token a token en CPU. Con Locust la proporción se mantiene considerando los tiempos de la mediana y el percentil noventa y ocho, puesto que los tiempos mínimos se alejan de la proporción anterior por la carga, que ralentiza al sistema y parece aumentar casi diez veces el resto de métricas.

Balanceo de carga. La estrategia `simple-shuffle` produjo una distribución prácticamente equitativa: 45 peticiones a ollama-1 (51 %) y 44 a ollama-2 (49 %). La pequeña asimetría es esperable en un reparto aleatorio con un número finito de muestras. Como solicitamos que se emplee el caché en este grupo, es comprensible que sus tiempos sean altos pero con poca variación, ya que la petición es idéntica cada vez.

7. Conclusiones

7.1. Cumplimiento de objetivos

Se ha construido y evaluado una granja de modelos de IA completamente funcional que integra balanceo de carga y caché semántico. El sistema se despliega de forma reproducible con un único comando (`docker compose up -d`) y expone una API compatible con OpenAI a través de un proxy Nginx.

Los resultados experimentales demuestran que:

- El caché semántico proporciona una aceleración de $191.5\times$ en latencia para preguntas repetidas o parafraseadas (31 ms frente a 5929 ms), eliminando la necesidad de ejecutar inferencia redundante.
- La detección de paráfrasis funciona correctamente en dominios con vocabulario bien diferenciado (Geografía, 100 %; Historia, 70 %), aunque presenta limitaciones en dominios técnicos como Ciencias (40 %).
- El balanceo de carga mediante `simple-shuffle` distribuye las peticiones de forma equitativa (51 %/49 %) sin necesidad de lógica compleja.
- El sistema puede mantener un funcionamiento y velocidad correctos de aproximadamente dos peticiones por segundo con cargas moderadas (aunque repetidas) con tan solo dos instancias.
- La arquitectura basada en Docker Compose permite escalar horizontalmente añadiendo más instancias de Ollama, modificar el umbral de similitud o cambiar el modelo sin alterar el resto del sistema.

7.2. Limitaciones

El sistema presenta varias limitaciones que conviene señalar:

- El umbral de similitud 0.85 es un valor fijo que no se adapta al dominio. Un umbral más bajo (0.80) habría capturado las paráfrasis de ciencia, pero podría haber generado más falsos positivos en otros grupos.
- La latencia base de inferencia (~6 s) está limitada por la ejecución en CPU. El uso de GPU aceleraría significativamente tanto la inferencia como la generación de embeddings.
- El sistema solo cuenta con dos réplicas de inferencia. Si ambas estuvieran saturadas, las peticiones se encolarían sin más recursos disponibles.
- El caché semántico introduce una latencia adicional en las peticiones MISS, ya que debe generar el embedding y consultar Redis antes de enviar la pregunta al modelo.

7.3. Trabajo futuro

A partir de los resultados obtenidos, se identifican las siguientes líneas de mejora:

1. **Umbral de similitud adaptativo:** ajustar dinámicamente el umbral en función del dominio detectado en la pregunta o de la densidad de entradas en el caché, minimizando tanto falsos positivos como falsos negativos.
2. **Caché multinivel:** combinar caché exacto (primer nivel, latencia < 1 ms) con caché semántico (segundo nivel, latencia ~31 ms), derivando al modelo solo cuando ambos fallan.
3. **Más réplicas de inferencia:** añadir instancias adicionales de Ollama e implementar escalado

dinámico basado en la carga observada.

4. **Aceleración por hardware:** evaluar el rendimiento con GPU para inferencia y generación de embeddings, reduciendo la latencia base de ~6 s a potencialmente cientos de milisegundos.
5. **Monitorización:** integrar Prometheus y Grafana para visualizar en tiempo real la tasa de acierto de caché, latencia por percentil y distribución de carga.
6. **Nuevo test de carga:** podemos rehacer/extender el *locustfile.py* para; en un entorno con más instancias; ejecutar una nueva prueba de carga, con lo que podríamos hacernos una idea de los límites del conjunto ampliado e identificar otros aspectos a mejorar o necesidades futuras, de los que tan solo podemos atisbar algo ahora.

Además, existen posibles mejoras en el campo de red:

1. **Reglas:** se pueden añadir medidas para eliminar el tráfico innecesario (IPTABLES, firewall, configuración de router...).
2. **Certificado SSL:** podríamos dotar a Nginx con este y así gozar de conexión cifrada, alterando su configuración.
3. **Optimización:** para mejorar la eficiencia, se pueden tener en cuenta características de la red como tamaño de paquetes y ancho de banda, y tomar medidas que mejoren el rendimiento medio.

Bibliografía

- [1] “Docker overview”. Accedido: Jun. 2026, Docker Inc. (n.d.), dirección: <https://docs.docker.com/get-started/docker-overview/>.
- [2] “Compose Specification”. Accedido: Jun. 2026, Docker Inc. (n.d.), dirección: <https://docs.docker.com/reference/compose-file/>.
- [3] “Ollama — Get up and running with large language models”. Accedido: Jun. 2026, Ollama. (n.d.), dirección: <https://ollama.com/>.
- [4] “Qwen3.5 — Model Card”. Accedido: Jun. 2026, Alibaba Cloud. (n.d.), dirección: <https://ollama.com/library/qwen3.5>.
- [5] “nomic-embed-text — Model Card”. Accedido: Jun. 2026, Nomic AI. (n.d.), dirección: <https://ollama.com/library/nomic-embed-text>.
- [6] “LiteLLM — Call all LLM APIs using the OpenAI format”. Accedido: Jun. 2026, BerriAI. (n.d.), dirección: <https://docs.litellm.ai/>.
- [7] “Redis Semantic Cache — LiteLLM Documentation”. Accedido: Jun. 2026, BerriAI. (n.d.), dirección: https://docs.litellm.ai/docs/caching/redis_cache.
- [8] “Redis Stack — Vector database”. Accedido: Jun. 2026, Redis Ltd. (n.d.), dirección: <https://redis.io/docs/latest/develop/get-started/vector-database/>.
- [9] Wikipedia contributors. “Cosine similarity”. Accedido: Jun. 2026. (n.d.), dirección: https://en.wikipedia.org/wiki/Cosine_similarity.
- [10] “NGINX Reverse Proxy — ngx_http_proxy_module”. Accedido: Jun. 2026, F5 Inc. (n.d.), dirección: https://nginx.org/en/docs/http/ngx_http_proxy_module.html.
- [11] C. Byström, J. Heyman y L. Holmberg. “Documentación de Locust”. Accedido: Jun. 2026. (s.f.), dirección: <https://docs.locust.io/en/stable/>.